

GAME NETWORKING – LEC/LAB

LESSON 6 - MIDTERM MINI PROJECT

MIDTERM MINI PROJECT:

Learning Objectives:

- **Knowledge (Cognitive):**
 - **Analyze** client-server and peer-to-peer network topologies, identifying state synchronization requirements, bandwidth costs, and security risks.
 - **Formulate** structured network protocols (RPCs, state snapshots, network variables) to maintain world consistency across variable latencies.
- **Skills (Psychomotor / Application):**
 - **Construct** functional 2-player networked mechanics in Unity (Netcode/Mirror) or Godot 4 (ENet/MultiplayerAPI) using custom Blender 3D assets.
 - **Implement** authoritative server validation, client-side prediction, interpolation, and session recovery routines.
- **Attitudes (Affective):**
 - **Appreciate** the importance of anti-cheat design, ethical multiplayer session management, and fair team play incentives.
 - **Commit** to professional industry software practices, clean network architecture, and collaborative peer code reviews.

Business Scenario:

Your indie game studio has completed the core single-player mechanics and 3D art pipeline in Blender. The publisher has commissioned your two-person technical team to build and integrate a specialized **Network Feature Module**. The module must operate under real-world network conditions (handling latency spikes, jitter, packet loss, and sudden disconnects) while ensuring authoritative data validation on the server to prevent local client tampering.

Lab Description:

Working in assigned pairs (Student A and Student B), student teams will select one mini-project title. Using their existing 3D game assets built in **Blender** and their chosen game engine (**Unity** with Netcode/Mirror or **Godot 4** with High-Level Multiplayer API), teams will engineer, test, and document an authoritative networked system. The prototype must connect two separate client instances over a local loopback (127.0.0.1) or LAN, complete with interactive UI feedback, server verification, and network error handling.

Date: September 14, 2026

Expected Outcome:

- A fully functional multiplayer feature module integrated into the team's custom 3D game.
- Zero desynchronization during standard movement and interaction loops across two connected game clients.
- Authoritative server handling that successfully rejects invalid or cheated client inputs.
- Seamless disconnect handling and UI updates during network dropouts.

Milestone 1 List of Deliverables (Submitted to MS Teams)

1. **Source Code & Network Framework (Zip/Repo Link):** Initial engine project containing working connection handshakes, lobby/staging scenes, and custom network manager scripts.
2. **Blender Source Files (.blend & .fbx/.glTF):** All custom 3D models and props prepared for the mini-project module with configured pivot points and colliders.
3. **Milestone 1 Architecture Document (PDF - 1 to 2 Pages):**
 - **Project Title & Team Roles:** Breakdown of Student A vs. Student B responsibilities.
 - **Network Data Flow Diagram:** Initial diagram detailing RPC routing or network variable declarations for their chosen title.
 - **Connection Test Log:** Screenshots verifying two executable windows connecting to the local server/host.

Milestone 2: Feature Completion, Lag Mitigation & Midterm Presentation

Submission Deadline: Monday, September 28, 2026 (via MS Teams)

Midterm Exam Event: Live Demo Presentation during class on September 28, 2026.

Goal: Complete feature module mechanics, implement server-authoritative validation and disconnect recovery, and present a live dual-window demonstration.

Objectives for Students

- Finalize the core mechanics specific to their selected mini-project title (e.g., dynamic trap sync, score ledgers, or speed-hack checks).
- Implement network lag mitigation (prediction, interpolation) and disconnect/recovery routines.
- Prepare a 5-minute live demonstration for the Midterm Examination.

Date: September 14, 2026

Milestone 2 List of Deliverables (Submitted to MS Teams):

1. **Final Source Code & Executable Builds:** Cleaned project folder and compiled standalone desktop build (.exe or executable package) capable of running multi-instance testing.
2. **Dual-Window Video Recording (2 to 3 Minutes):** High-quality backup video showing side-by-side execution of the feature, artificial lag simulation (150ms ping), and client disconnect handling.
3. **Final Technical Architecture Report (PDF - 3 to 4 Pages):**
 - Comprehensive data flow documentation detailing server-authoritative validation logic.
 - Edge-case test matrix (handling packet loss, sudden disconnects, invalid inputs).
 - Final post-mortem reflection on pair programming contributions.

Adjusted Midterm Grading Distribution:

Evaluation Component	Weight	Due Date	Description
Milestone 1 Deliverables	25%	Sept 21, 2026	Network architecture setup, Blender 3D integration, and connection proof.
Milestone 2 Deliverables	40%	Sept 28, 2026	Completed feature mechanics, network lag mitigation code, and technical PDF report.
Midterm Live Demo & Defense	35%	Sept 28, 2026	Live dual-window demonstration, real-time disconnect recovery, and Q&A defense.

These 20 titles are organized into 5 functional networking categories (4 projects per category):

Category A: Authoritative Mechanics & Physics Sync

- **Goal Addressed: Consistency & Security (Server Authority)**
- **Why It Matters:** In games like Portal 2 or Fall Guys, if clients were allowed to dictate their own physics positions, a player with a modified client could teleport across the map or bypass traps. By requiring the server to calculate moving platforms, traps, and projectiles, the game guarantees that physical interactions are 100% fair and synchronized across all peer screens.

1. **Project 1: Authoritative Dynamic Trap & Hazard System**

- **Focus:** Server-validated environmental hazards (e.g., swinging blades, collapsing floors) with client-side prediction and hit validation.
- **Game Example:** *Fall Guys* or *It Takes Two*
- **Context:** Swinging pendulums, rotating beams, and rising lava are processed on the server to ensure all players see the exact same hazard timing and hitboxes.

2. **Project 2: Networked 3D Physics Puzzle Switches & Pressure Plates**

- **Focus:** Synchronizing rigidbodies, pushable 3D Blender blocks, and multi-weighted switches validated by the authoritative host.
- **Game Example:** *Portal 2 (Co-Op)* or *Human: Fall Flat*
- **Context:** Carrying weighted cubes and placing them onto pressure plates where physics calculations and door state updates are validated by the host server.

3. **Project 3: Synchronized Co-Op Projectile & Spell-Casting Pipeline**

- **Focus:** Event-based RPC trajectory syncing, collision prediction, and area-of-effect (AoE) server verification.
- **Game Example:** *Magicka* or *Risk of Rain 2*
- **Context:** Players casting spell projectiles or throwing energy Orbs where position trajectory and splash damage hitboxes are synced via RPCs.

4. **Project 4: Networked Moving Platforms & Shared Vehicle Transport**

- **Focus:** Hierarchical transform synchronization, attaching player character controllers to moving networked platforms without visual jitter.
- **Game Example:** *Sea of Thieves* or *Unravel Two*
- **Context:** Standing on a moving boat, elevator, or floating platform without lagging off or jittering while moving across the world space.

Category B: Session Management, Lobbies & Matchmaking

- **Goal Addressed: Operational Stability & User Onboarding**
- **Why It Matters:** A game can have perfect in-game movement, but if players cannot reliably connect, select roles without race conditions, or survive a host drop, the multiplayer experience fails. Session management

Date: September 14, 2026

forms the bridge between offline menus and active multiplayer worlds, ensuring clean lifecycle handshakes and host migration.

5. **Project 5: Private Room Code Matchmaking & Custom Password System**

- *Focus:* Building a secure matchmaking staging system using unique 6-digit room keys, host privileges, and player filtering.
- **Game Example:** *Among Us* or *Phasmophobia*
- **Context:** Generating a 6-letter join code (e.g., ROOM-X7) so friends can join private lobby sessions directly without public matchmaking.

6. **Project 6: Networked Character Selection & Dynamic Loadout Locker**

- *Focus:* Real-time 3D model skin/role swapping in the lobby (e.g., Lumen vs. Firefly) with race-condition prevention on slot locking.
- **Game Example:** *Overwatch 2* or *Apex Legends*
- **Context:** Locking in character roles in a pre-game staging screen where picking a character locks out that slot to prevent partner duplicates.

7. **Project 7: Asynchronous Mid-Game Reconnection & State Catch-Up**

- *Focus:* Allowing a player who dropped mid-session to rejoin the active room and re-sync their state without restarting the level.
- **Game Example:** *Left 4 Dead 2* or *Deep Rock Galactic*
- **Context:** If a player crashes mid-mission, they can rejoin the active session and automatically sync to the current objective state without restarting.

8. **Project 8: Host Migration & Server Authority Handover Module**

- *Focus:* Detecting host disconnection, pausing game state, promoting the client to host, and resuming simulation seamlessly.
- **Game Example:** *Call of Duty (Peer-to-Peer matches)* or *Warframe*
- **Context:** When the party host disconnects, the session pauses briefly with a "Migrating Host..." message, promotes another peer to server host, and resumes seamlessly.

Category C: Cooperative & Competitive Scoring Systems

- **Goal Addressed: Data Integrity & Balanced Incentive**
- **Why It Matters:** Shared team pools, timers, and inventory ledgers must be protected from race conditions (e.g., both players grabbing the last key item at the exact same millisecond). Processing transactions through authoritative RPCs ensures items aren't duplicated and score achievements are accurately awarded.

9. Project 9: Authoritative Hybrid Co-Opetition Score Ledger & MVP System

- **Focus:** Server-side tracking of shared team points vs. individual synergy bonuses with an end-of-level dynamic leaderboard.
- **Game Example:** *Lego Star Wars (Co-Op)* or *Helldivers 2*
- **Context:** Completing shared mission objectives while simultaneously tracking individual player stats (e.g., Accuracy, Assists, MVP rank) on an end-of-level screen.

10. Project 10: Synchronized Puzzle Timer & Penalty Reduction Engine

- **Focus:** Server-authoritative global countdown timers, network clock synchronization, and team penalty distribution.
- **Game Example:** *Keep Talking and Nobody Explodes* or *Escape Academy*
- **Context:** A synchronized ticking countdown clock on the HUD where triggering a trap deducts 30 seconds from both players' shared time pool.

11. Project 11: Co-Op Resource Management & Shared Inventory Ledger

- **Focus:** Networked item pickups (shards, keys), inventory transaction RPCs, and drop/trade state sync between 3D characters.
- **Game Example:** *Don't Starve Together* or *Valheim*
- **Context:** Picking up items from the 3D environment, trading inventory items between players, and updating a shared bank/chest network state.

12. Project 12: Real-Time Team Health, Resuscitation & Defibrillation Sync

- **Focus:** Synchronized downs/revives mechanics, channel timers, and shared revive pools across network clients.
- **Game Example:** *The Division 2* or *Monster Hunter: World*
- **Context:** When a teammate takes critical damage, they enter a "downed" state, allowing their partner to hold an interact key for 3 seconds to revive them.

Category D: Network Lag Mitigation & Anti-Cheat Validation

- **Goal Addressed: Perceived Responsiveness & Bandwidth Efficiency**
- **Why It Matters:** Information cannot travel faster than the speed of light. Because round-trip latency (ping) is unavoidable, mechanics like **client-side prediction** allow local players to feel zero input delay, while **interpolation** hides jitter for remote players. Concurrently, distance-based net-culling ensures the server only transmits data players actually need, keeping packet sizes lean.

13. Project 13: Client-Side Movement Prediction & Server Reconciliation Module

- *Focus:* Implementing local input prediction with server rollback buffers for fast-paced 3D character movement.
- **Game Example:** *Rocket League* or *Valorant*
- **Context:** Local player input moves the character instantly to feel responsive, while the server verifies movement vectors asynchronously to prevent teleportation exploits.

14. Project 14: Networked Interpolation & Jitter Buffer Smoothing System

- *Focus:* Linear and spherical interpolation (Lerp/Slerp) of remote entity positions under 150ms simulated ping and 10% packet loss.
- **Game Example:** *Counter-Strike 2* or *Fortnite*
- **Context:** Remote players glide smoothly across the screen even when their connection experiences high ping spikes or minor packet drops.

15. Project 15: Authoritative Speed-Hack & Teleportation Prevention System

- *Focus:* Server distance-delta and velocity validation checks that rubber-band clients attempting invalid movement speeds.
- **Game Example:** *World of Warcraft* or *Fall Guys*
- **Context:** Server checks maximum allowable distance over time ($\Delta d / \Delta t$) and forces a rubber-band rollback if a client tries to modify local speed values.

16. Project 16: Dynamic Interest Management & Distance-Based Net-Culling

- *Focus:* Optimizing network bandwidth by selectively pausing updates for 3D entities located outside the player's view frustum or proximity radius.
- **Game Example:** *Rust* or *DayZ*
- **Context:** Disabling network updates for objects and players located far away from the local character to save network bandwidth and processing performance.

Category E: Advanced Multiplayer Mechanics & Interactive World State

- **Goal Addressed: Multi-User Synchronicity & World Consistency**
- **Why It Matters:** Complex interactions—such as two players pulling a lever at the exact same frame or blowing up a 3D wall—require tight timestamp validation. This category ensures environmental changes and AI state machines reflect identical conditions across all client displays simultaneously.

17. Project 17: Multi-User Shared Object Manipulation & Dual-Key Doors

- **Focus:** Requiring two clients to trigger actions within a tight server-validated time window (e.g., turning two valves simultaneously).
- **Game Example:** *A Way Out* or *It Takes Two*
- **Context:** Both players must press a button or turn a valve wheel at the exact same time (within a 0.5-second server-validated window) to open a heavy door.

18. Project 18: Networked Dynamic Environment & Destruction Synchronization

- **Focus:** Syncing Blender destructible mesh states, mesh swapping, and debris spawn events across host and client.
- **Game Example:** *Rainbow Six Siege* or *Battlefield*
- **Context:** Blowing up a wall or breaking a wooden barricade so that structural destruction and debris spawn in sync across all clients.

19. Project 19: Authoritative AI Boss & Minion Aggro Sync Engine

- **Focus:** Server-driven AI state machines running pathfinding (NavMesh) and syncing target aggro changes to all peers.
- **Game Example:** *Dark Souls (Co-Op)* or *Remnant: From the Ashes*
- **Context:** A 3D boss enemy managed by the host server that switches target focus (aggro) between players based on who dealt the most recent damage.

20. Project 20: In-Game Spatial Voice/Text Chat & Networked Ping Indicator

- **Focus:** 3D spatialized communication, quick contextual ping alerts (3D world markers), and server RPC chat broadcasting.
- **Game Example:** *Apex Legends* or *Lethal Company*
- **Context:** Pressing a ping button to place a 3D marker in the world space ("*Look Here!*") visible to teammates alongside spatial chat channels.

Evaluation Rubric:

This unified rubric will be used to grade all 20 mini-projects submitted via MS Teams.

Criteria	Weight	Excellent (4)	Proficient (3)	Developing (2)	Novice (1)
Networking Architecture & Server Authority	30%	Mechanics are 100% server-authoritative. RPCs and network variables are structured cleanly with zero vulnerability to client-side manipulation.	Server authority handles core gameplay, but minor edge cases allow clients to bypass validation checks.	Network logic is split inconsistently; clients directly mutate shared states without server verification.	Code is purely client-driven or lacks functional multiplayer networking.
State Synchronization & Lag Smoothing	25%	Entities update smoothly across screens with no visual rubber-banding, using prediction or interpolation under simulated lag.	State sync works under ideal network conditions, but exhibits minor jitter during latency spikes.	Frequent desynchronization between player screens; objects drop out of sync or stutter severely.	Network states fail to synchronize entirely between host and client.
Asset Integration & 3D Implementation	15%	Custom 3D assets designed in Blender are properly rigged, textured, and integrated seamlessly with network components in Unity/Godot.	3D assets render correctly in multiplayer, but show minor visual bugs or unoptimized mesh colliders.	Basic 3D asset usage with missing textures, incorrect origin pivots, or broken collision hulls.	Fails to incorporate 3D assets or uses unmodified engine primitive blocks only.

Date: September 14, 2026

Criteria	Weight	Excellent (4)	Proficient (3)	Developing (2)	Novice (1)
Error Handling & Disconnect Recovery	15%	App handles unexpected client drops, network timeouts, or invalid join requests gracefully with UI popups and clean memory teardown.	Disconnects are detected, but recovery leaves minor console warnings or requires manual room restart.	Dropping a connection soft-locks the game scene or throws unhandled runtime exceptions.	No disconnect detection or network error handling implemented.
Deliverables & Team Documentation	15%	Submission includes clean code, .blend sources, high-quality dual-window video demo, and a detailed architecture PDF report.	Complete submission, but video demo lacks lag simulation or documentation misses structural diagrams.	Incomplete submission (missing video, Blender files, or architectural breakdown).	Missing major deliverables or submitted past deadline without prior clearance.

Due date: September 21 and 28, 2026